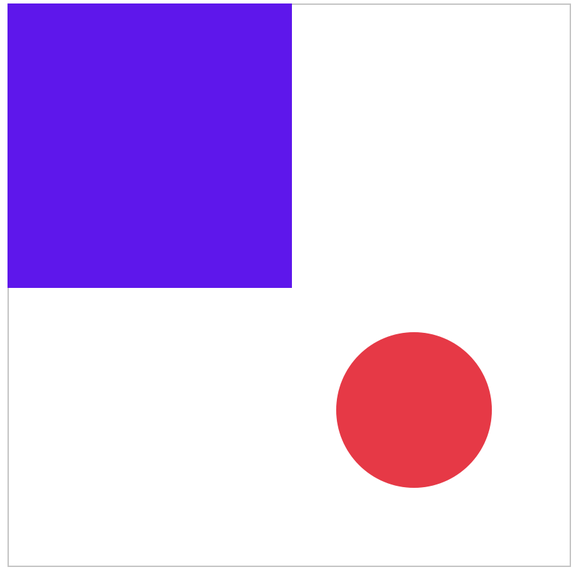
Reducing False Payment *Declines*



Fares Tinakiche

PRODUCT MANAGER L1

Answers section 1 of the brief

The Situation

We run an online payment gateway that merchants use to take payments on their websites. Six weeks ago, it started declining customers who had done nothing wrong.

The case

Six Weeks Ago, Payments Started Failing

CONTEXT

Merchants integrate our online payment gateway into their e-commerce websites to process customer payments.

WHAT IS HAPPENING

Over the past **6 weeks**, our monitoring dashboard has shown a significant increase in transactions classified as fraudulent.

MERCHANTS & SUPPORT REPORT

Customers are unable to complete their payments

Merchants have lower checkout conversion rates and are losing sales

INITIAL FINDING

A large proportion of transactions flagged as fraudulent were later confirmed to be legitimate customer purchases.

Over the past 6 weeks

Who It Hurts, and How

CUSTOMER

- Legitimate payments are being declined
- Customers retry or abandon checkout

MERCHANT

- Lost sales
- Lower confidence in the payment gateway

BUSINESS

- Checkout conversion ↓
- Partner bank fraud risk scores ↑
- Processing fees ↑

PRODUCT GOAL

Reduce false declines while maintaining strong fraud protection.

The Key Change: Hosted Checkout Became the Default

BEFORE

Merchant-controlled checkout

Customer

↓

Merchant Checkout

↓

Payment Gateway API

↓

Bank / Fraud System

6 WEEKS AGO

Hosted Checkout introduced as default

Customer

↓

Payment Gateway Hosted Checkout

↓

Bank / Fraud System

WHY THIS MATTERS

The payment company now controls the payment experience and recovery UX. The bank still controls the external fraud decision.

Answers section 2 of the brief

Finding the Cause

Four explanations fit the symptoms. Only one of them is ours to chase first.

What the Evidence Actually Says

01 Fraud classification is performed by external partner bank systems.

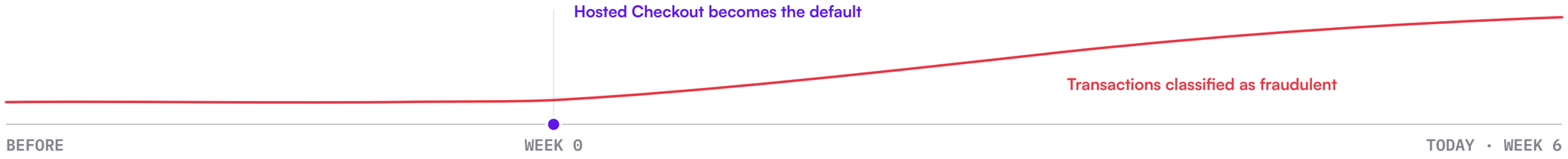
02 Banks consider multiple payment attempts a suspicious signal.

03 Banks consider repeated card-detail changes during one checkout session suspicious.

04 A large proportion of transactions flagged as fraudulent are later confirmed legitimate.

05 Hosted Checkout became the default around the same time the problem appeared.

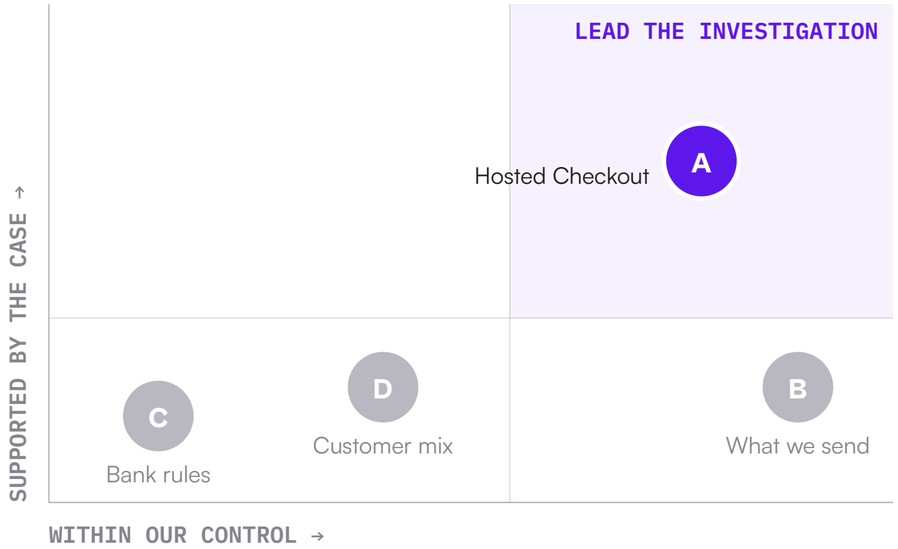
Fraud classifications, before and after Hosted Checkout FROM THE CASE · TIMING EVIDENCE ONLY



CORRELATION ≠ CAUSATION

I would validate the relationship before committing to a solution.

Four Explanations, One to Chase First



A ★ Our checkout changed how people pay

It went live when the problem started, and the behaviour it creates is what the banks act on.

B The information we send the bank changed

The hosted page may send different device, address or merchant details. Banks score on those.

C The bank changed its rules

Possible, but we would have to ask them. We cannot see their model from here.

D Different customers or merchants

Possible, and cheap to check. We split the data and see if it sits in one group.

Hypothesis A, in specifics

What Might Hosted Checkout Be Doing?

01	The error message does not say what went wrong, so the customer guesses and tries again
02	The form clears after a decline, so they re-type everything, which the bank reads as changed card details
03	Nothing stops a double-click, so one click can send two attempts
04	The checkout session times out mid-payment, so they start over

THESE ARE CANDIDATES, NOT FINDINGS

Each one is something I would check in the flow during validation.

WHY THESE FOUR

Every one of them produces one of the two behaviours the partner banks penalise: more attempts in a short window, or changed card details in a single session.

Before Building: Validate the Root Cause

01

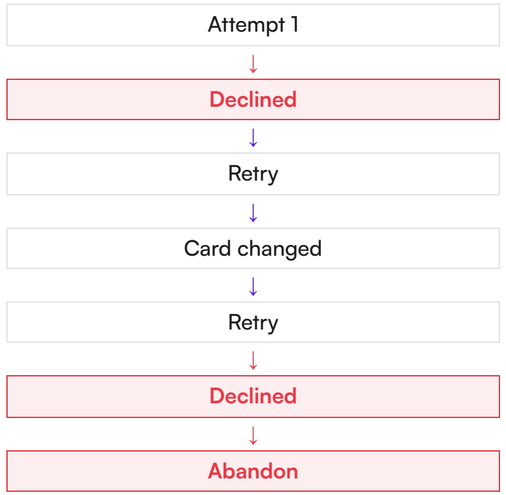
Before vs. After Hosted Checkout

Compare: previous 6 weeks ↔ current 6 weeks

- **Payment success rate**
  How many customers complete payment successfully?
- **False decline rate**
  Is the problem getting worse?
- **Retry rate**
  Are customers attempting more than once?
- **Card-change rate**
  Are they swapping card details?
- **Attempts per session**
  How intense is the retry behaviour?

02

Analyze the Payment Journey



Count in both: merchant checkout → Hosted Checkout

Question: Did this pattern increase after Hosted Checkout?

03

Segment the Data

Analyze by:

`Bank` `Merchant` `Payment Method` `Country` `Device` `Card Type`

Goal: Identify where the problem is concentrated.

04

Talk to People

- **Customers and merchants:** why they retry or swap cards, and where checkout confuses them
- **Data / Analytics:** whether our false-decline number is measured correctly
- **Partner Banks:** how many retries they tolerate before flagging

VALIDATION DECISION

If the data, user behavior and stakeholder feedback consistently point to Hosted Checkout and repeated payment attempts, proceed with the proposed recovery solution.

Answers section 3 of the brief

The Fix

We cannot change how the bank scores a payment. We can change what our checkout sends it.

What I'm Assuming

01

What we know

The banks tell us that too many payment attempts in a short time looks suspicious.

They don't say how many.

02

What I assume

4 attempts in 5–10 minutes

My number, not theirs. It is the first thing I would check against our data and with the banks.

03

What we do

Slow the customer down on the **3rd** attempt.

We act before the line is reached, and we ship it as a configurable policy rather than a hard-coded number.

THE SECOND THING I ASSUME

That Hosted Checkout does neither of these today: no clear decline reason with a next step, and no 3-D Secure step-up on a retry. Validation answers that in a day. If it already does one of them, that part of the fix drops and the retry controls carry the work.

The Recovery Flow, Before and After

CURRENT EXPERIENCE

Payment Declined

↓

Customer doesn't know what happened

↓

Retry

↓

Change Card

↓

Retry Again

↓

Repeated attempts + card changes

↓

Potential false decline

WHAT 3-D SECURE COSTS US

The challenge is friction and some customers will drop at it, and passing it does not override the bank's own fraud decline. I would test it with partner banks before relying on it.

PROPOSED EXPERIENCE

Payment Declined

↓

Clear reason + next best action

Retry payment | Use another card

↓

Repeated failures + card changes*

→ review the new payment details

↓

Repeated failures (after 3rd attempt)

↓

Next attempt requests a 3-D Secure challenge

the bank asks the customer to confirm it is them

↓ ↓ ↓

Approved	Still declined	Not supported
payment completes	cool down*, or another payment method	cool down*, or another payment method

What Changes

PRODUCT CHANGES

PROBLEM	PRODUCT RESPONSE
Customers don't understand why a payment failed	Clear reason + next best action
Customers change card details repeatedly	Ask the customer to review the new details before submitting again
One click can send two payment attempts	One click, one payment. A double-click or a refresh never reaches the bank twice
Customers retry rapidly, again and again	Request a 3-D Secure (SCA) challenge on the retry, not on the first attempt
The payment is declined even after verifying	Short cooldown, so we stop adding suspicious attempts
A legitimate customer still wants to pay	Choose another payment method, checkout session preserved

Customers are failing now, so the mitigation does not wait for validation

Protect Conversion While Fixing the Root Cause

TRACK 1 · MITIGATE NOW

Route the worst-hit merchants and customers back to API Checkout
The dashboard already shows where the declines sit, so this does not wait for the full analysis

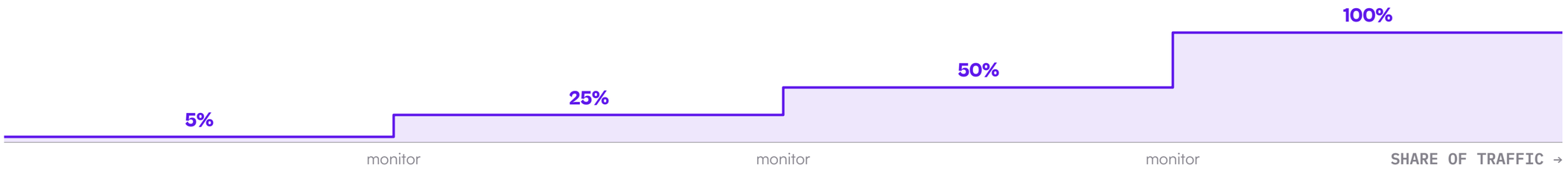
fallback removed once the fix is live

TRACK 2 · FIX THE CAUSE

Validate the cause	Fix Hosted Checkout	Experiment	Scale
Is it really our checkout?	Clear reasons, card and bank checks, recovery paths	Improved against current	5% to 100%, gated

NOW → TIME →

ROLLOUT



We start at 5% to catch anything broken, then only grow while the numbers hold.

Benchmarks and Stakeholders

BENCHMARKS

- **Our own numbers before the change**
  The six weeks before Hosted Checkout, our only internal baseline
- **Industry decline rates**
  Is our rate normal for online card payments, or unusual?
- **How other payment providers handle declines**
  What they show customers, and how they allow retries
- **Partner bank documentation**
  How many retries they tolerate before flagging

STAKEHOLDERS

INTERNAL

- **Data / Analytics team:** measurement and experiment readout
- **Engineering team:** checkout, retry logic, feature flags
- **Product design team:** failure messaging and recovery flows

EXTERNAL

- **Merchants:** affected by every change we ship to checkout
- **Partner Banks:** own the fraud models and our risk score

Answers section 4 of the brief

Proving It Works

Two numbers I want to move, two that must not get worse, two that tell me early whether it is working, and two that only move on the banks' timetable.

How Will We Know It Worked?

WHAT WE WANT TO MOVE

False decline rate ↓
Are fewer real customers being rejected?

Payment success rate ↑
Are more legitimate payments going through?

WHAT MUST NOT GET WORSE

Actual fraud ↔ / ↓
Are we still catching real fraud?

Payment errors ↔ / ↓
Did we break anything technically?

WHAT TELLS US EARLY

Attempts per session ↓
The behaviour the banks penalise

Card-change rate ↓
The second signal the banks watch

BUSINESS METRICS

Processing fees ↓
The real payoff, but on the banks' timetable

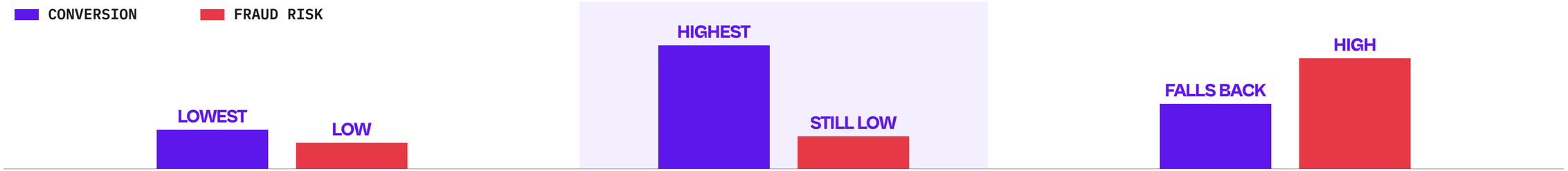
Our risk score with the banks ↓
Reviewed quarterly with Partnerships, not inside the experiment

MY EARLY READ

If attempts per session has not moved in 48 hours, I do not wait for the false-decline number. The fix is not doing its job.

The Line We Must Not Cross

Conversion peaks before fraud risk starts climbing CONCEPTUAL · RELATIVE LEVELS, NOT MEASURED DATA



TOO RESTRICTIVE

Strict retry limits, so legitimate customers cannot recover

BALANCED RECOVERY

Controlled retries with clear guidance, so customers recover without extra signals

TOO PERMISSIVE

Unlimited retries, so more suspicious attempts reach the bank and get declined

WHAT COULD GO WRONG

The bank still decides

We can fix our checkout and the bank can still say no. That part is not ours to control.

Customers dropping at the bank check

Some will abandon at the 3-D Secure challenge, and some banks may not support it at all. That is why it only fires on the 3rd attempt, and why we measure how many finish it.

Moving every merchant twice

Rolling everyone back and forward again is disruptive. We only do it while the fix is being built, and we remove the fallback the day it is live.

THE TRADE-OFF INSIDE OUR OWN FIX

Verification and cooldowns are friction, and friction costs conversion. We accept it only on the tail of sessions already heading for a decline. A short wait beats a hard refusal.

Answers section 5 of the brief

After Launch

The work that decides whether this succeeded happens after launch.

What I Watch, and When

The numbers from the last slide, placed by when each one becomes trustworthy.

FIRST 24 HOURS

Is anything broken?

- Payment errors
- Fraud alerts
- Payment success rate

Why now: these three show up straight away. If one of them goes wrong, I stop the rollout the same day.

FIRST 7 DAYS

Did behaviour change?

- Attempts per session
- Card-change rate

Why now: a week is enough to see retry behaviour change.

14–30 DAYS

Did the banks change their answer?

- False decline rate
- Actual fraud

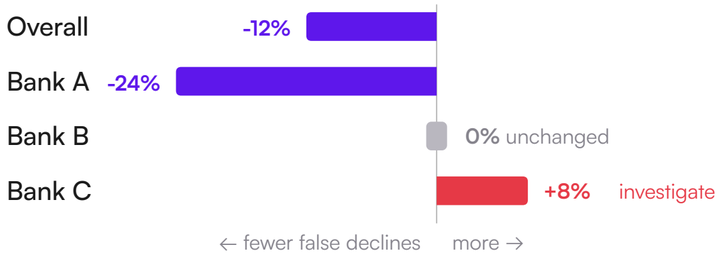
Then cut it by:

`Bank` `Merchant` `Country` `Payment Method`

Why now: both take weeks to confirm, because chargebacks arrive late. It is also the first point with enough volume to cut by bank.

EXAMPLE

Change in false decline rate by partner bank ILLUSTRATIVE



→ Investigate Bank C separately.

In this order, so the cheap checks come first

Check These In Order Before Changing the Plan

01 Check the numbers are right
Are we counting false declines the same way we did before?

02 Check the behaviour changed
Did retries and card changes drop? If they did not, the fix is not working yet.

03 Check what the bank did
Retries can drop while declines stay flat. That gap is the bank's side.

04 Then decide
Keep fixing the flow, roll it back, or go back to the other explanations.

Beyond monitoring the metrics

What I Own After Release

01 Share the results
Tell merchants, Risk, Data and leadership what happened, including what did not improve.

02 Go back to the banks
Show them the better false-decline numbers and ask them to review our risk score and our fees.

03 Turn off the workaround
Remove the API Checkout fallback and the temporary flags so they do not become permanent.

04 Change how we ship defaults
This started because a default went live without a test behind it. The next one gets one.

My Product Approach

01 · VALIDATE

Prove whether Hosted Checkout changed retry and card-change behavior.

02 · UNDERSTAND

Combine transaction data with customer, merchant, Risk and bank feedback.

03 · MITIGATE

Route the worst-hit merchants back to API Checkout temporarily, until the hosted page is fixed.

04 · FIX

Improve Hosted Checkout recovery: clear messaging, controlled retries, session preservation, alternative payment methods, a risk-aware 3-D Secure step-up.

05 · EXPERIMENT

Compare improved Hosted Checkout against the current experience.

06 · SCALE

5% → 25% → 50% → 100%. Only scale if: False Declines ↓ • Payment Success ↑ • Actual Fraud ↔ / ↓

Reduce unnecessary fraud-like behavior without making legitimate customers pay the price.